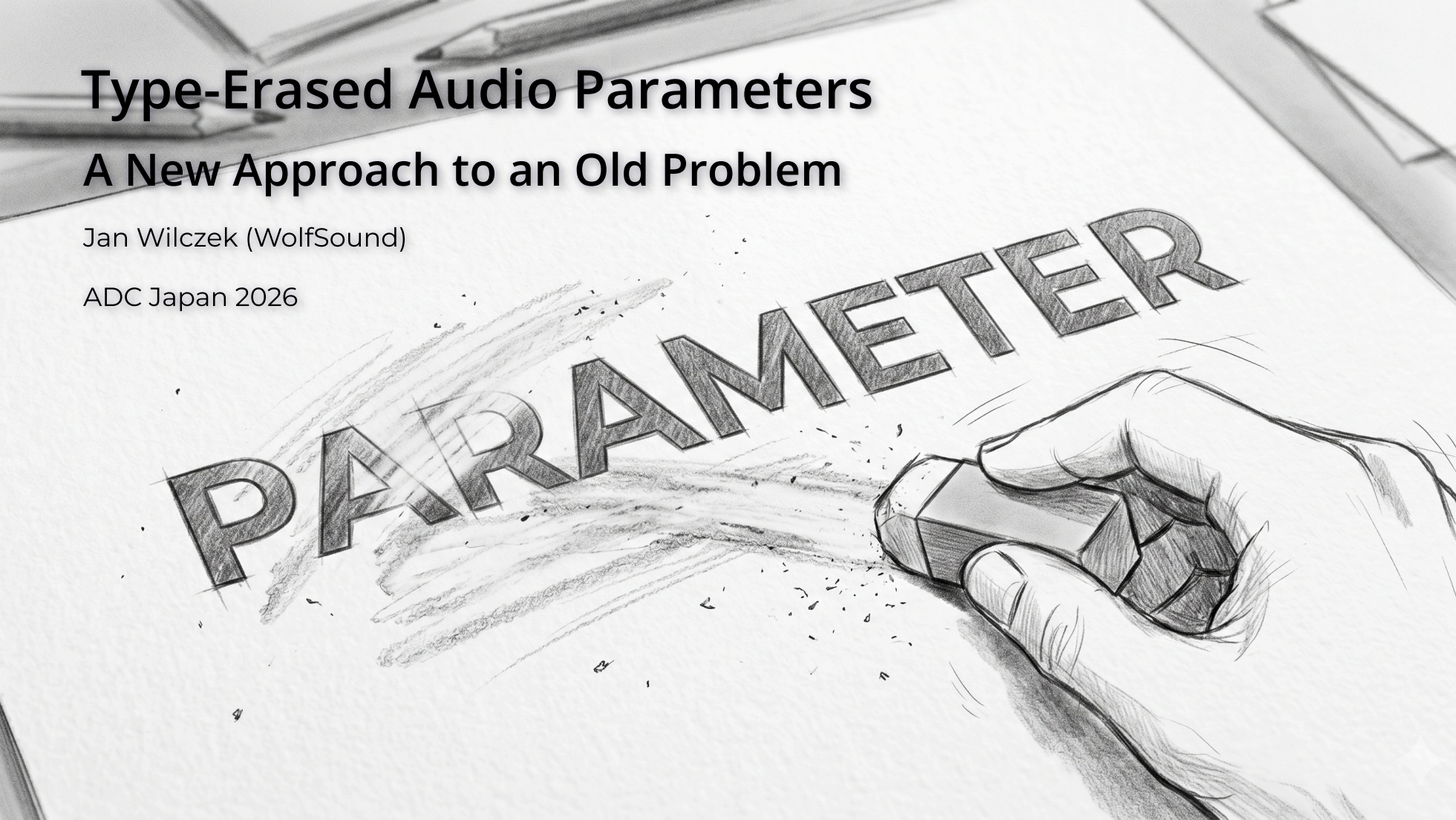
Type-Erased Audio Parameters

A New Approach to an Old Problem

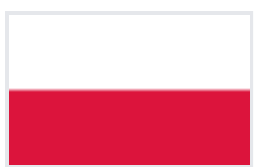
Jan Wilczek (WolfSound)

ADC Japan 2026



Who am I?

- Jan Wilczek [Yan Vil-check]
- Audio programming consultant & educator
- Founder of TheWolfSound.com blog & YouTube channel on audio programming
- WolfTalk podcast host
- Trainer
  - conference workshops
  - in-house training on DSP/JUCE
- Online course creator
  - DSP Pro on digital audio signal processing
  - Official JUCE C++ framework audio plugin development course (over 4,400 students enrolled)



Source: https://en.wikipedia.org/wiki/Japan-Poland_relations, accessed May 31, 2026.

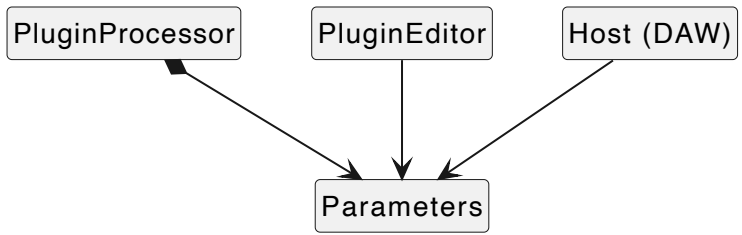
Who here develops or uses audio plugins for digital audio workstations?

Who here uses JUCE to develop plugins?

Key terms

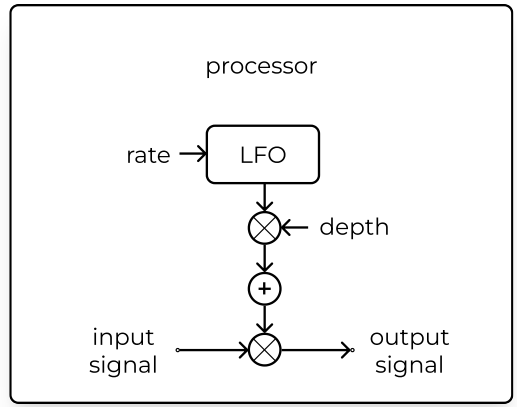
- **audio plugin**: a plugin for a digital audio workstation (DAW)
- **plugin processor**: the core of an audio plugin; responsible for plugin metadata, audio processing, and state management
- **plugin editor**: user interface (UI) of the plugin
- **plugin parameter**: a user-controllable value influencing audio processing of a plugin
  - `juce::AudioParameterFloat|Bool|Int|Choice` class instance in JUCE plugins
- **UI state**: non-audio-related state
- **serialization**: process of externalizing state (for example, to a JSON file)
- **deserialization**: process of loading state from an external source (for example, a JSON file)
- **preset** = plugin parameter values + metadata

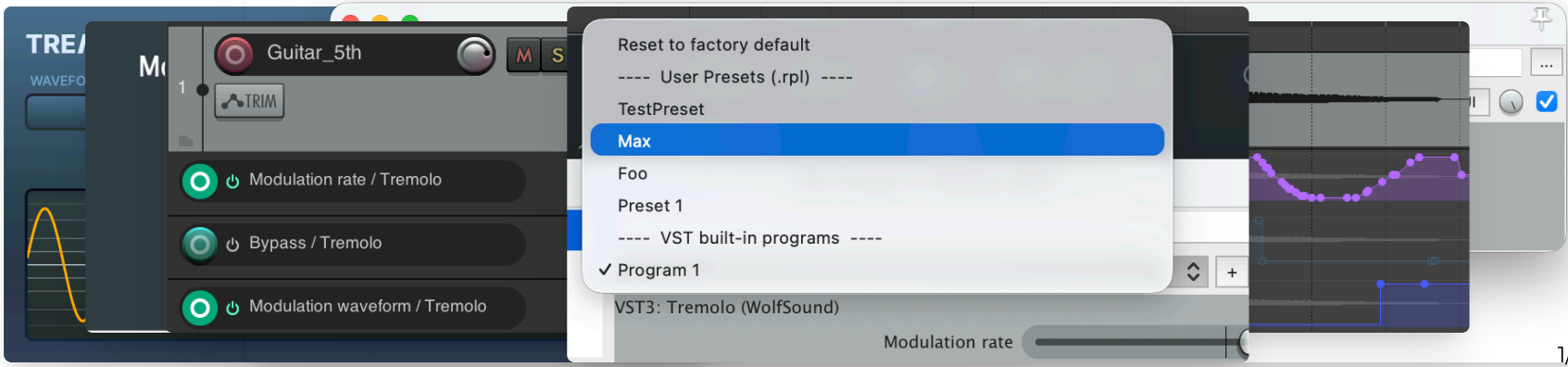
Parameters in audio plugins



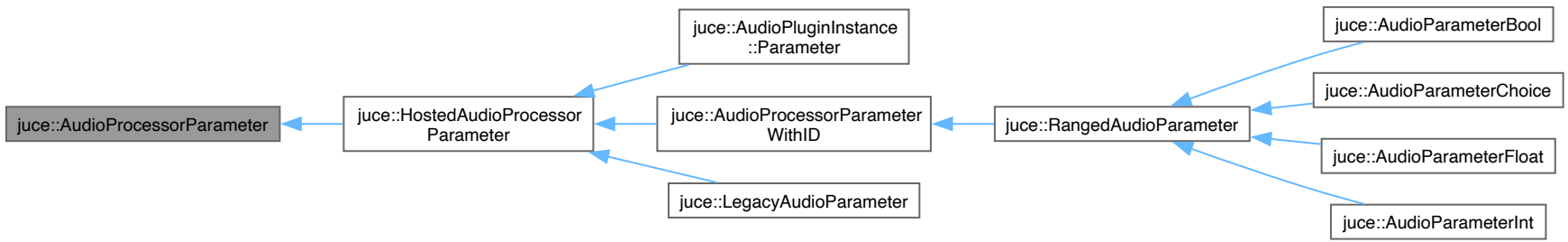
- Automation
- Serialization
- Presets (generic/custom)
- Visualization (generic/custom)
- UI (generic/custom)

Example of plugin parameters in action





Parameter class hierarchy in JUCE

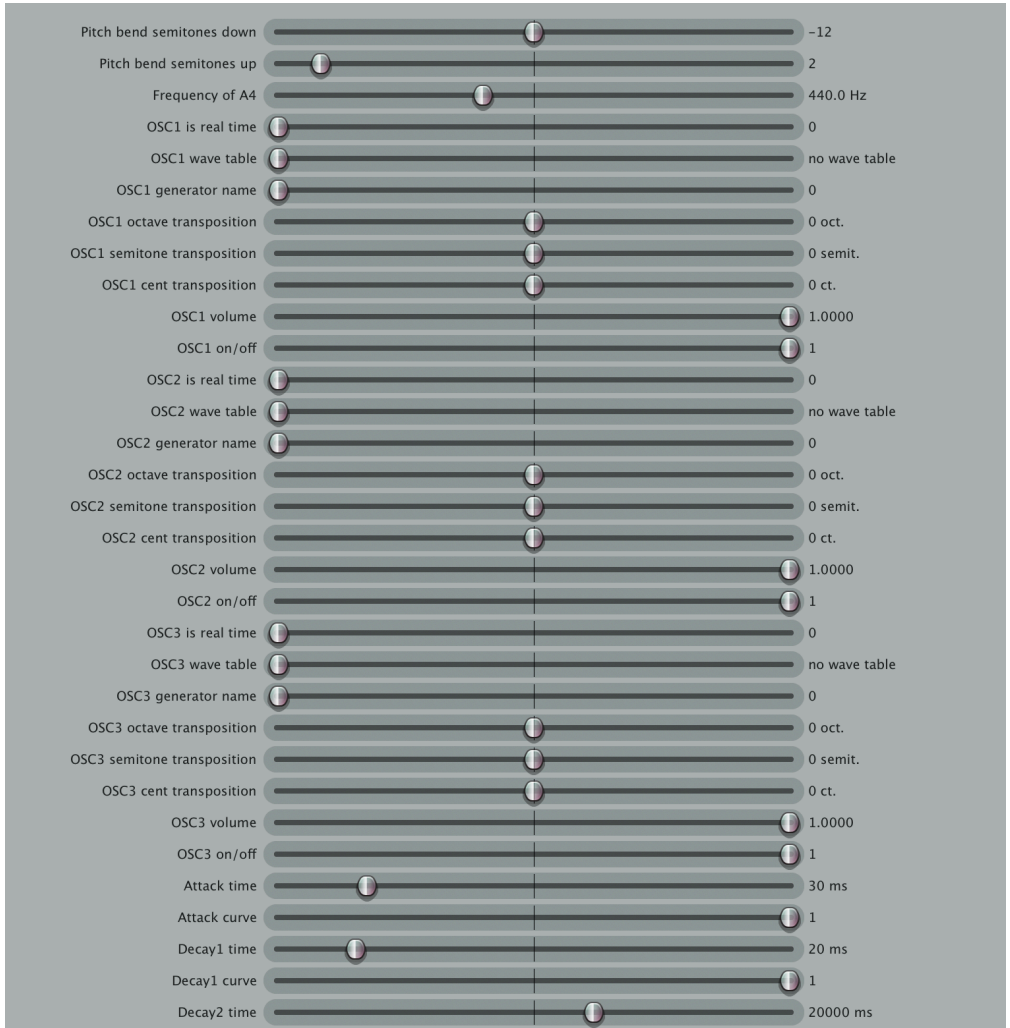


JUCE parameter classes

- `AudioParameterBool`
  - `true` / `false`
- `AudioParameterInt`
  - integer in a closed range
  - e.g., "1" from {0, 1, 2}
- `AudioParameterFloat`
  - real value from a closed range
  - e.g., "0.25" from [0, 2]
- `AudioParameterChoice`
  - a value from a fixed set of named options
  - e.g., "lowpass" from {"lowpass", "highpass"}

The Story of a Synth


Options
EdenSynth
Preset:
Save preset
Pitch bend up
Pitch bend down
A4 frequency
Octave
Transpose
Real time
Volume
White Noise
Octave
Transpose
Fine tune
Real time
Volume
Triangle
Octave
Transpose
Fine tune
Real time
Volume
Sine
Attack
Decay 1
Decay 2
Release
Break level
Linear
Linear
Linear
Linear
Cutoff
Resonance
Contour amount
Passband attenuation
24 dB
Attack
Decay
Sustain
Release
Volume
Transfer function
Hyperbolic ta...
Degree:
4
Spread


Pitch bend semitones down
-12
Pitch bend semitones up
2
Frequency of A4
440.0 Hz
OSC1 is real time
0
OSC1 wave table
no wave table
OSC1 generator name
0
OSC1 octave transposition
0 oct.
OSC1 semitone transposition
0 semit.
OSC1 cent transposition
0 ct.
OSC1 volume
1.0000
OSC1 on/off
1
OSC2 is real time
0
OSC2 wave table
no wave table
OSC2 generator name
0
OSC2 octave transposition
0 oct.
OSC2 semitone transposition
0 semit.
OSC2 cent transposition
0 ct.
OSC2 volume
1.0000
OSC2 on/off
1
OSC3 is real time
0
OSC3 wave table
no wave table
OSC3 generator name
0
OSC3 octave transposition
0 oct.
OSC3 semitone transposition
0 semit.
OSC3 cent transposition
0 ct.
OSC3 volume
1.0000
OSC3 on/off
1
Attack time
30 ms
Attack curve
1
Decay1 time
20 ms
Decay1 curve
1
Decay2 time
20000 ms

Plugin processor

```cpp
class EdenSynthAudioProcessor : public AudioProcessor {
public:
  EdenSynthAudioProcessor();
  ~EdenSynthAudioProcessor();

  void prepareToPlay(double sampleRate, int samplesPerBlock) override;
  void releaseResources() override;

#ifndef JucePlugin_PreferredChannelConfigurations
  bool isBusesLayoutSupported(const BusesLayout& layouts) const override;
#endif

  void processBlock(AudioBuffer<float>&, MidiBuffer&) override;

  AudioProcessorEditor* createEditor() override;
  bool hasEditor() const override;

  const String getName() const override;

  bool acceptsMidi() const override;
  bool producesMidi() const override;
  bool isMidiEffect() const override;
  double getTailLengthSeconds() const override;

  int getNumPrograms() override;
  int getCurrentProgram() override;
  void setCurrentProgram(int index) override;
  const String getProgramName(int index) override;
  void changeProgramName(int index, const String& newName) override;

  void getStateInformation(MemoryBlock& destData) override;
  void setStateInformation(const void* data, int sizeInBytes) override;

private:
  JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(EdenSynthAudioProcessor)

  std::filesystem::path _assetsPath;
  eden::EdenSynthesiser _edenSynthesiser;
  eden_vst::EdenAdapter _edenAdapter;
  AudioProcessorValueTreeState _pluginParameters;
};
```

Parameters via `AudioProcessorValueTreeState`

```
EdenSynthAudioProcessor::EdenSynthAudioProcessor()
    : // ...
      _pluginParameters{*this, nullptr} {
  using Parameter = juce::AudioProcessorValueTreeState::Parameter;

  _pluginParameters.createAndAddParameter(std::make_unique<Parameter>(
      "pitchBend.semitonesDown", "Pitch bend semitones down",
      NormalisableRange<float>(-24.f, 0.f, 1.f), -12.f));
  _pluginParameters.createAndAddParameter(std::make_unique<Parameter>(
      "pitchBend.semitonesUp", "Pitch bend semitones up",
      NormalisableRange<float>(0.f, 24.f, 1.f), 2.f));
  _pluginParameters.createAndAddParameter(std::make_unique<Parameter>(
      "frequencyOfA4", "Frequency of A4",
      NormalisableRange<float>(400.f, 500.f, 0.1f), 440.f,
      AudioProcessorValueTreeStateParameterAttributes{}.withLabel("Hz")));
  // more parameters ...

  _pluginParameters.state = ValueTree(Identifier("EdenSynthParameters"));
}
```

Parameters via `AudioProcessorValueTreeState`

New API

```
EdenSynthAudioProcessor::EdenSynthAudioProcessor()
    : // ...
      _pluginParameters{*this, nullptr, "EdenSynthParameters", {
          std::make_unique<AudioParameterFloat>(
              "pitchBend.semitonesDown", "Pitch bend semitones down",
              NormalisableRange<float>(-24.f, 0.f, 1.f), -12.f),
          std::make_unique<AudioParameterFloat>(
              "pitchBend.semitonesUp", "Pitch bend semitones up",
              NormalisableRange<float>(0.f, 24.f, 1.f), 2.f),
          std::make_unique<AudioParameterFloat>(
              "frequencyOfA4", "Frequency of A4",
              NormalisableRange<float>(400.f, 500.f, 0.1f), 440.f,
              AudioProcessorValueTreeStateParameterAttributes{}.withLabel("Hz")),
          // more parameters ...
      }} {}
```

Parameters via `AudioProcessorValueTreeState`

Usage in audio processing

```
1   void EdenSynthAudioProcessor::processBlock(AudioBuffer<float>& buffer,
2                                              MidiBuffer& midiMessages) {
3     // ...
4
5     _synthesiser.setPitchBendRange(
6         {static_cast<int>(
7              *pluginParameters.getRawParameterValue("pitchBend.semitonesDown")),
8          static_cast<int>(
9              *pluginParameters.getRawParameterValue("pitchBend.semitonesUp"))});
10    _synthesiser.setFrequencyOfA4(
11        *pluginParameters.getRawParameterValue("frequencyOfA4"));
12
13    // audio & MIDI processing
14  }
```

Parameters via `AudioProcessorValueTreeState`

Serialization

```
1  void EdenSynthAudioProcessor::getStateInformation(MemoryBlock& destData) {
2    auto state = _pluginParameters.copyState();
3    const std::unique_ptr<XmlElement> xml(state.createXml());
4    copyXmlToBinary(*xml, destData);
5  }
```

Parameters via `AudioProcessorValueTreeState`

Deserialization

```
void EdenSynthAudioProcessor::setStateInformation(const void* data,
                                                  int sizeInBytes) {
  std::unique_ptr<XmlElement> xmlState(getXmlFromBinary(data, sizeInBytes));

  if (xmlState.get()) {
    if (xmlState->hasTagName(_pluginParameters.state.getType())) {
      _pluginParameters.replaceState(ValueTree::fromXml(*xmlState));
    }
  }
}
```

Parameters via `AudioProcessorValueTreeState`

Usage in the plugin editor

```
class GeneralSettingsComponent : public Component {
public:
  using SliderAttachment = AudioProcessorValueTreeState::SliderAttachment;

  GeneralSettingsComponent(AudioProcessorValueTreeState&);

  void resized() override;
  void paint(Graphics& g) override;

private:
  Slider _pitchBendSemitonesUp;
  std::unique_ptr<SliderAttachment> _pitchBendSemitonesUpAttachment;

  Slider _pitchBendSemitonesDown;
  std::unique_ptr<SliderAttachment> _pitchBendSemitonesDownAttachment;

  Slider _a4Frequency;
  std::unique_ptr<SliderAttachment> _a4FrequencyAttachment;
};
```

Parameters via `AudioProcessorValueTreeState`

Usage in the plugin editor

```
GeneralSettingsComponent::GeneralSettingsComponent(
    AudioProcessorValueTreeState& valueTreeState)
    : _pitchBendSemitonesUp{/* */},
      _pitchBendSemitonesDown{/* */},
      _a4Frequency{/* */} {
  addAndMakeVisible(_pitchBendSemitonesUp);
  _pitchBendSemitonesUpAttachment = std::make_unique<SliderAttachment>(
      valueTreeState, "pitchBend.semitonesUp", _pitchBendSemitonesUp);

  addAndMakeVisible(_pitchBendSemitonesDown);
  _pitchBendSemitonesDownAttachment = std::make_unique<SliderAttachment>(
      valueTreeState, "pitchBend.semitonesDown", _pitchBendSemitonesDown);

  addAndMakeVisible(_a4Frequency);
  _a4FrequencyAttachment = std::make_unique<SliderAttachment>(
      valueTreeState, "frequencyOfA4", _a4Frequency);
}
```

How to add presets? 🤔

Reuse `get/setStateInformation()`

```
void savePreset(const std::string& presetName) {
    juce::MemoryBlock presetData;
    pluginProcessor.getStateInformation(result);
    const auto presetFile = juce::File{presetPathFromName(name)};
    presetFile.deleteFile();
    presetFile.appendData(presetData.getData(), presetData.getSize());
}
```

Reuse `get/setStateInformation()`

```
1   void loadPreset(const std::string& presetName) {
2     const auto presetFile = juce::File{presetPathFromName(name)};
3
4     juce::MemoryBlock presetData;
5     const auto result = presetFile.loadFileAsData(presetData);
6
7     if (!result) {
8       return;
9     }
10
11    pluginProcessor.setStateInformation(presetData.getData(),
12                                        static_cast<int>(presetData.getSize()));
13  }
```

Preset structure when using APVTS

```xml
<?xml version="1.0" encoding="UTF-8"?>

<EdenSynthParameters>
  <PARAM id="envelope.adbdr.attack.curve" value="1.0" />
  <PARAM id="envelope.adbdr.attack.time" value="30.0" />
  <PARAM id="envelope.adbdr.breakLevel" value="0.60000002384185791" />
  <PARAM id="envelope.adbdr.decay1.curve" value="1.0" />
  <PARAM id="envelope.adbdr.decay1.time" value="20.0" />
  <PARAM id="envelope.adbdr.decay2.curve" value="1.0" />
  <PARAM id="envelope.adbdr.decay2.time" value="20000.0" />
  <PARAM id="envelope.adbdr.release.curve" value="1.0" />
  <PARAM id="envelope.adbdr.release.time" value="300.0" />
  <PARAM id="filter.contourAmount" value="1.0" />
  <PARAM id="filter.cutoff" value="1.0" />
  <PARAM id="filter.passbandAttenuation" value="0.0" />
  <PARAM id="filter.resonance" value="0.0" />
  <PARAM id="frequencyOfA4" value="440.0" />
  <!-- more parameters ... -->
  <PARAM id="output.volume" value="1.0" />
  <PARAM id="pitchBend.semitonesDown" value="-12.0" />
  <PARAM id="pitchBend.semitonesUp" value="2.0" />
  <PARAM id="waveshaper.autoMakeUpGain" value="0.0" />
</EdenSynthParameters>
```

APVTS-based parameters

Pros

- Easy to implement
- Automatic serialization of all parameters for free
- Can work (somewhat) for presets
- Easy UI attachments

Cons

- Parameters mangled with UI state
- Little control over XML structure in serialization
- Values as `float`s (no meaningful values or types)
  - `processBlock()`
  - presets
- What if I want a different serialization format, like JSON?
- Other APVTS drawbacks, e.g., requires a message thread

What we did in the official JUCE course

"References only"

Parameters via references only

```
class PluginProcessor : public juce::AudioProcessor {
public:
    PluginProcessor();
    // ...
    struct Parameters {
        explicit Parameters(juce::AudioProcessor&);
        juce::AudioParameterFloat& rate;
        juce::AudioParameterBool& bypassed;
        juce::AudioParameterChoice& waveform;
    };
    Parameters parameters{*this};
};
```

Parameters via references only

```cpp
namespace {
auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
  auto& result = *parameter;
  processor.addParameter(parameter.release());
  return result;
}

juce::AudioParameterFloat& createModulationRateParameter(
    juce::AudioProcessor& processor) {
  constexpr auto versionHint = 1;
  return addParameterToProcessor(
      processor,
      std::make_unique<juce::AudioParameterFloat>(
          juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
          juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
          juce::AudioParameterFloatAttributes{}.withLabel("Hz")));
}

juce::AudioParameterBool& createBypassedParameter(
    juce::AudioProcessor& processor) {
  constexpr auto versionHint = 1;
  return addParameterToProcessor(
      processor,
      std::make_unique<juce::AudioParameterBool>(
          juce::ParameterID{"bypassed", versionHint}, "Bypass", false));
}

juce::AudioParameterChoice& createWaveformParameter(
    juce::AudioProcessor& processor) {
  constexpr auto versionHint = 1;
  return addParameterToProcessor(
      processor,
      std::make_unique<juce::AudioParameterChoice>(
          juce::ParameterID{"modulation.waveform", versionHint},
          "Modulation waveform", juce::StringArray{"Sine", "Triangle"}, 0));
}
} // namespace

Parameters::Parameters(juce::AudioProcessor& p)
    : rate{createModulationRateParameter(p)},
      bypassed{createBypassedParameter(p)},
      waveform{createWaveformParameter(p)} {}
```

Parameters via references only

```
namespace {
auto& addParameterToProcessor(juce::AudioProcessor& processor, auto parameter) {
  auto& result = *parameter;
  processor.addParameter(parameter.release());
  return result;
}

juce::AudioParameterFloat& createModulationRateParameter(
    juce::AudioProcessor& processor) {
  constexpr auto versionHint = 1;
  return addParameterToProcessor(
      processor,
      std::make_unique<juce::AudioParameterFloat>(
          juce::ParameterID{"modulation.rate", versionHint}, "Modulation rate",
          juce::NormalisableRange<float>{0.1f, 20.f, 0.01f, 0.4f}, 5.f,
          juce::AudioParameterFloatAttributes{}.withLabel("Hz")));
}
} // namespace

Parameters::Parameters(juce::AudioProcessor& p)
    : rate{createModulationRateParameter(p)},
        /* ... */ {}
```

Parameters via references only

Usage in audio processing

```
1   void PluginProcessor::processBlock(juce::AudioBuffer<float>& buffer,
2                                      juce::MidiBuffer&) {
3     // ...
4     tremolo.setModulationRateHz(parameters.rate.get());
5     tremolo.setLfoWaveform(
6         static_cast<Tremolo::LfoWaveform>(parameters.waveform.getIndex()));
7     bypassTransitionSmoother.setBypass(parameters.bypassed.get());
8
9     // audio processing
10  }
```

Parameters via references only

Usage in UI

```
class PluginEditor : public juce::AudioProcessorEditor {
public:
  explicit PluginEditor(PluginProcessor&);

  void resized() override;

private:
  juce::ComboBox waveformComboBox;
  juce::ComboBoxParameterAttachment waveformAttachment;

  juce::Slider rateSlider;
  juce::SliderParameterAttachment rateAttachment;

  juce::ToggleButton bypassButton{"BYPASSED"};
  juce::ButtonParameterAttachment bypassAttachment;
};
```

Parameters via references only

Usage in UI

```
1   PluginEditor::PluginEditor(PluginProcessor& p)
2       : AudioProcessorEditor(&p),
3         waveformAttachment{p.parameters.waveform, waveformComboBox},
4         rateAttachment{p.parameters.rate, rateSlider},
5         bypassAttachment{p.parameters.bypassed, bypassButton} {}
```

Parameters via references only

Serialization

```
void PluginProcessor::getStateInformation(juce::MemoryBlock& destData) {
  juce::MemoryOutputStream outputStream{destData, true};
  JsonSerializer::serialize(parameters, outputStream);
}

void PluginProcessor::setStateInformation(const void* data, int sizeInBytes) {
  juce::MemoryInputStream inputStream{data, static_cast<size_t>(sizeInBytes),
                                      false};
  const auto result = JsonSerializer::deserialize(inputStream, parameters);
  if (result.failed()) {
    // notify the user
  }
  // optionally skip smoothing
}
```

Parameters via references only

Serialization

```
struct SerializableParameters {
  float rate;
  bool bypassed;
  juce::String waveform;

  static constexpr auto marshallingVersion = 1;

  template <typename Archive, typename T>
  static void serialise(Archive& archive, T& p) {
    using namespace juce;

    if (archive.getVersion() != 1) {
      return;
    }

    std::string pluginName = TREMOLO_PLUGIN_NAME;

    archive(named("pluginName", pluginName));

    if (pluginName != TREMOLO_PLUGIN_NAME) {
      return;
    }

    archive(named("modulationRateHz", p.rate), named("bypassed", p.bypassed),
            named("modulationWaveform", p.waveform));
  }
};
```

```
SerializableParameters from(const Parameters& p) {
  return {
      .rate = p.rate.get(),
      .bypassed = p.bypassed.get(),
      .waveform = p.waveform.getCurrentChoiceName(),
  };
}

void JsonSerializer::serialize(const Parameters& parameters,
                               juce::OutputStream& output) {
  const auto json = juce::ToVar::convert(from(parameters));

  if (!json.has_value()) {
    return;
  }

  juce::JSON::writeToStream(output, *json,
                            juce::JSON::FormatOptions{}
                                .withSpacing(juce::JSON::Spacing::multiLine)
                                .withMaxDecimalPlaces(2));
}
```

Concrete-type based parameters

Pros

- Full type safety
- Easy access to singular parameters
  - `processBlock()`
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization

Cons

- "Manual" serialization code
  - Adding new parameters requires updating `JsonSerializer` $\Longrightarrow$ error-prone

Summary so far

1. We can treat plugin parameters as a collection of `juce::RangedAudioParameter`s (just like `juce::AudioProcessorValueTreeState`) $\implies$ We lose type information
2. We can treat plugin parameters individually using only concrete `juce::AudioParameterFloat|Bool|Int|Choice` classes $\implies$ We cannot (easily) define operations on a collection of parameters

Summary so far

1. Parameter collection: extensibility
2. Individual parameters: interpretability

How can we treat parameters as a collection without losing type information? 🤔

Answer: Type Erasure!

Type-Erased Parameters

Collection of parameters

```
1   class TypeErasedParameter;
2
3   std::vector<TypeErasedParameter> parameters;
```

TypeErasedParameter

```
class TypeErasedParameter {
public:
    template <class Parameter>
    TypeErasedParameter(Parameter& p) : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}

private:
    class ParameterConcept {
    public:
        virtual ~ParameterConcept() = default;
    };

    template <class Parameter>
    class ParameterModel : public ParameterConcept {
    public:
        ParameterModel(Parameter& p) : _p{p} {}
    private:
        Parameter& _p;
    };

    std::unique_ptr<ParameterConcept> _impl;
};

std::vector<TypeErasedParameter> parameters;
```

Operations

```
void serializeToJson(juce::AudioParameterFloat& p);
void serializeToJson(juce::AudioParameterBool& p);
// ...
class TypeErasedParameter {
public:
    // ...
    void serializeToJson() { _impl->serializeToJson(); }
private:
    class ParameterConcept {
    public:
        virtual ~ParameterConcept() = default;
        virtual void serializeToJson() = 0;
    };
    template <class Parameter>
    class ParameterModel : public ParameterConcept {
    public:
        // ...
        void serializeToJson() override { serializeToJson(_p); }

    private:
        Parameter& _p;
    };
    std::unique_ptr<ParameterConcept> _impl;
};
```

Serialization

```
struct Serializer {
    virtual ~Serializer = default;
    virtual void serializeToJson(juce::AudioParameterFloat& p) = 0;
    virtual void serializeToJson(juce::AudioParameterBool& p) = 0;
    // ...
};

class TypeErasedParameter {
public:
    // ...
    void serialize(Serializer& s) { _impl->serialize(s); }
private:
    class ParameterConcept {
    public:
        virtual ~ParameterConcept() = default;
        virtual void serialize(Serializer&) = 0;
    };
    template <class Parameter>
    class ParameterModel : public ParameterConcept {
    public:
        // ...
        void serialize(Serializer& s) override { serializer.serialize(_p); }

    private:
```

Operations supporting all JUCE parameter classes

```
struct Visitor {
    virtual ~Visitor = default;
    virtual void visit(juce::AudioParameterFloat& p) = 0;
    virtual void visit(juce::AudioParameterBool& p) = 0;
    // ...
};

class TypeErasedParameter {
public:
    // ...
    void accept(Visitor& v) { _impl->accept(v); }
private:
    class ParameterConcept {
    public:
        virtual ~ParameterConcept() = default;
        virtual void accept(Vistior&) = 0;
    };
    template <class Parameter>
    class ParameterModel : public ParameterConcept {
    public:
        // ...
        void accept(Visitor& v) override { v.visit(_p); }

    private:
```

Collection of `TypeErasedParameter`s

```
1   std::vector<TypeErasedParameter> parameters;
```

Collection of `TypeErasedParameter`s

```
class ParameterHolder {
  class TypeErasedParameter {
  public:
    template <class Parameter>
    explicit TypeErasedParameter(Parameter& p)
        : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}

    void accept(Visitor& v) { _impl->accept(v); }

  private:
    class ParameterConcept {  // NOLINT
    public:
      virtual ~ParameterConcept() = default;
      virtual void accept(Visitor& v) = 0;
    };

    template <class Parameter>
    class ParameterModel : public ParameterConcept {
    public:
      explicit ParameterModel(Parameter& p) : _p{p} {}

      void accept(Visitor& v) override { v.visit(_p.get()); }

    private:
      Parameter& _p;
    };

    std::unique_ptr<ParameterConcept> _impl;
  };

public:
  explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
      : _parameters{std::move(parameters)} {}

  void accept(Visitor& v) {
    for (auto& parameter : _parameters) {
      parameter.accept(v);
    }
  }

private:
  std::vector<TypeErasedParameter> _parameters;
};
```

Builder

```
class Builder {
public:
    template <class P, class ... Args>
    P& add(Args&& ... args) {
        auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
        auto& ref = *parameter;
        _parametersForHolder.emplace_back(ref);
        _parameters.push_back(std::move(parameter));
        return ref;
    }

    ParameterHolder build(juce::AudioProcessor& p) && {
        for (auto&& parameter : _parameters) {
            p.addParameter(parameter.release());
        }
        return ParameterHolder{std::move(_parametersForHolder)};
    }

private:
    std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
    std::vector<TypeErasedParameter> _parametersForHolder;
};
```

Builder

```cpp
class ParameterHolder {
  class TypeErasedParameter {
  public:
    template <class Parameter>
    explicit TypeErasedParameter(Parameter& p)
        : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
        // ...
  };

public:
  class Builder {
  public:
    template <class P, class... Args>
    P& add(Args&&... args) {
      auto parameter = std::make_unique<P>(std::forward<Args>(args)...);
      auto& ref = *parameter;
      _parametersForHolder.emplace_back(ref);
      _parameters.push_back(std::move(parameter));
      return ref;
    }

    ParameterHolder build(juce::AudioProcessor& p) && {
      for (auto&& parameter : _parameters) {
        p.addParameter(parameter.release());
      }
      return ParameterHolder{std::move(_parametersForHolder)};
    }

  private:
    std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
    std::vector<TypeErasedParameter> _parametersForHolder;
  };

  void accept(Visitor& v) {/* ... */}

private:
  explicit ParameterHolder(std::vector<TypeErasedParameter> parameters)
      : _parameters{std::move(parameters)} {}

  std::vector<TypeErasedParameter> _parameters;
};
```

Usage

```
1   class PluginProcessor : public juce::AudioProcessor {
2   public:
3     explicit PluginProcessor(
4         ParameterHolder::Builder builder = {})
5         : floatParam{builder.add<juce::AudioParameterFloat>(
6               "floatParam", "Float Param", juce::NormalisableRange{1.f, 10.f}, 5.f)},
7           boolParam{builder.add<juce::AudioParameterBool>(
8               "boolParam", "Bool Param", true)},
9           intParam{builder.add<juce::AudioParameterInt>(
10              "intParam", "Int Param", 5, 10, 6)},
11          choiceParam{builder.add<juce::AudioParameterChoice>(
12              "choiceParam", "Choice Param", juce::StringArray{"choice 0", "choice 1", "choice 2"}, 1)},
13          parameterHolder{std::move(builder).build(*this)} {}
14
15    // ...
16  private:
17    juce::AudioParameterFloat& floatParam;
18    juce::AudioParameterBool& boolParam;
19    juce::AudioParameterInt& intParam;
20    juce::AudioParameterChoice& choiceParam;
21    ParameterHolder parameterHolder;
22  };
```

Serialization using a Visitor

```cpp
struct ParameterIdAndValue {
  std::string id;
  std::variant<float, int, bool, std::string> value;
};

class ParameterValuesExtractor : public Visitor {
public:
  ParameterValuesExtractor() = default;
  void visit(juce::AudioParameterFloat& parameter) override { visitImpl(parameter, parameter.get()); }
  void visit(juce::AudioParameterBool& parameter) override { visitImpl(parameter, parameter.get()); }
  void visit(juce::AudioParameterInt& parameter) override { visitImpl(parameter, parameter.get()); }
  void visit(juce::AudioParameterChoice& parameter) override {
      visitImpl(parameter, parameter.getCurrentChoiceName().toStdString());
  }
  [[nodiscard]] std::vector<ParameterIdAndValue> result() const { return _result; }
private:
  template <class P, class V>
  void visitImpl(const P& parameter, V&& value) {
    _result.emplace_back(parameter.getParameterID().toStdString(), std::forward<V>(value));
  }

  std::vector<ParameterIdAndValue> _result;
};
```

Serialization using a Visitor

```
1  std::vector<ParameterIdAndValue> parameterIdsAndValues(ParameterHolder& ph) {
2    ParameterValuesExtractor visitor;
3    ph.accept(visitor);
4    return visitor.result();
5  }
```

What if we want to support custom parameter classes?

$\Longrightarrow$ make `ParameterHolder` templated on the `Visitor` class.

What if we want to support custom parameter classes?

```
1  template <class Visitor>
2  class ParameterHolder {
3    // ...
4  public:
5    void accept(Visitor& v) {/*  ...  */}
6    // ...
7  };
```

What if we want to support custom parameter classes?

A good default

```
1  struct JuceParameterVisitor {
2    virtual ~JuceParameterVisitor() = default;
3    virtual void visit(juce::AudioParameterBool&) = 0;
4    virtual void visit(juce::AudioParameterFloat&) = 0;
5    virtual void visit(juce::AudioParameterInt&) = 0;
6    virtual void visit(juce::AudioParameterChoice&) = 0;
7  };
8
9  using JuceParameterHolder = ParameterHolder<JuceParameterVisitor>;
```

Implementations

`ParameterHolder` with example serialization

- https://github.com/JanWilczek/wolfsound-dsp-utils
  - *src/include/wolfsound/juce/wolfsound_ParameterHolder.hpp*

`ParameterHolder` with serialization and presets (WIP)

- https://github.com/JanWilczek/EdenSynth/tree/add-xml-presets-macos-var-params
  - *EdenSynth/SharedCode/include/presets/Preset.h*
  - *EdenSynth/SharedCode_test/source/presets_test/PresetsTest.cpp*

References

1. JUCE C++ framework source code, *https://github.com/juce-framework/JUCE*
2. Kevlin Henney, *Valued Conversions*, *C++ Report* July-August 2000
3. Sean Parent, *Inheritance Is the Base Class of Evil*, GoingNative 2013
4. Klaus Iglberger, *C++ Software Design: Design Principles and Patterns for High-Quality Software*, O'Reilly 2022
5. Jan Wilczek & the JUCE team, *Official JUCE Audio Plugin Development Online Course*, *https://wolfsoundacademy.com/juce* (available for free)

Summary

1. General: Use the Type Erasure design pattern together with the Visitor design pattern to manage collections of strongly typed objects of different classes
2. Specific: Use Type Erasure to perform operations on all parameter objects of your plugin without losing their type
    - Keep references to concrete parameter objects to access them individually
3. Try out/tweak `wolfsound::ParameterHolder` from *github.com/JanWilczek/wolfsound-dsp-utils*
4. Get slides at *github.com/JanWilczek/adc26-japan-talk*
5. Contact me via contact@thewolfsound.com